Лабораторная работа №2. Управление потоками

Вихарев Лев Александрович, М8О-214БВ-25. Вариант 3.

Цель и задание

Приобрести навыки управления потоками и их синхронизации. Отсортировать массив целых чисел параллельной сортировкой слиянием на языке C, ограничив количество потоков ключом программы. Исследовать ускорение и эффективность при изменении входных данных и числа потоков.

Алгоритм и синхронизация

Реализована рекурсивная сортировка слиянием. Функция merge_sort делит диапазон пополам, сортирует обе половины и объединяет их функцией merge. Границы диапазона задаются полуинтервалом [left, right). Вспомогательный массив выделяется один раз. Результат слияния сначала записывается во вспомогательный массив, затем копируется в исходный.

В parallel_sort левая половина может передаваться новому потоку, а правая обрабатывается текущим. После pthread_join выполняется слияние. Счётчик free_threads изначально равен P−1; перед созданием потока слот резервируется под защитой pthread_mutex, после его завершения и ожидания слот освобождается. Ожидающий родитель тоже считается потоком, его слот не освобождается. Поэтому одновременно существует не больше P потоков, включая главный. Для диапазона короче 16384 элементов и при отсутствии свободного слота используется последовательная сортировка. Ошибки pthread завершают весь процесс с диагностикой.

Разные потоки обрабатывают непересекающиеся участки обоих массивов. Мьютекс защищает только счётчик слотов, а pthread_join обеспечивает готовность половины перед слиянием. Время сортировки O(n log n), дополнительная память O(n) для буфера и O(P log n) для стеков потоков в верхней оценке. Финальное слияние выполняется одним потоком, что ограничивает ускорение.

Запуск и проверка

Сборка в Linux: make. Запуск: ./merge THREADS [SIZE] [random|sorted|reverse]. Пример: ./merge 4 1000000. Если размер не задан, создаются 5 000 000 элементов. Данные генерируются автоматически с srand(42); два одинаковых массива сортируются последовательно и параллельно. При размере не больше 20 выводятся исходные и отсортированные элементы. Размер и число потоков задаются положительными целыми числами; верхняя граница числа потоков — 256. Ручной ввод элементов не используется.

Для демонстрации потоков: python3 tools/check_threads.py. Скрипт наблюдает число потоков через /proc и выводит снимок ps -T. При лимитах 1, 2, 3, 4 и 8 наблюдались соответствующие максимумы без превышений. Проверка make test: 45 сравнений с независимой qsort, включая пустой массив на уровне функций, границы int и размеры вокруг порога распараллеливания; 22 проверки CLI, включая неверные аргументы. Все проверки прошли.

Исследование

Среда: Ubuntu 24.04 в WSL2, Intel Core i5-13420H, системе доступно 12 логических процессоров. Сборка GCC с оптимизацией -O2. Для каждой комбинации выполнены три запуска; отдельно взяты медианы времени последовательной и параллельной сортировки по CLOCK_MONOTONIC. Генерация, выделение памяти, вывод и проверка результата в это время не входят. Ускорение S = median(Tseq)/median(Tpar), эффективность E = S/p. В каждом запуске последовательная сортировка выполняется первой; прогрев и нагрузка системы могут влиять на результаты. Полные замеры сохранены в measurements.csv, повторный запуск: python3 tools/bench.py.

Размер массива: 10000

Данные	Потоки	Tseq, с	Tpar, с	S	E
случайные	1	0.000544	0.000540	1.006	1.006
случайные	2	0.000530	0.000581	0.911	0.456
случайные	4	0.000571	0.000566	1.008	0.252
случайные	8	0.000539	0.000551	0.977	0.122
по возрастанию	1	0.000112	0.000116	0.964	0.964
по возрастанию	2	0.000127	0.000134	0.943	0.471
по возрастанию	4	0.000111	0.000128	0.866	0.217
по возрастанию	8	0.000116	0.000128	0.906	0.113
по убыванию	1	0.000104	0.000110	0.949	0.949
по убыванию	2	0.000105	0.000106	0.997	0.499
по убыванию	4	0.000390	0.000103	3.798	0.949
по убыванию	8	0.000105	0.000137	0.767	0.096

Размер массива: 1000000

Данные	Потоки	Tseq, с	Tpar, с	S	E
случайные	1	0.086408	0.086149	1.003	1.003
случайные	2	0.081620	0.043417	1.880	0.940
случайные	4	0.082710	0.043137	1.917	0.479
случайные	8	0.078442	0.025066	3.129	0.391
по возрастанию	1	0.018549	0.016729	1.109	1.109
по возрастанию	2	0.017314	0.008688	1.993	0.996
по возрастанию	4	0.017991	0.008585	2.096	0.524
по возрастанию	8	0.016805	0.006251	2.688	0.336
по убыванию	1	0.018014	0.016302	1.105	1.105
по убыванию	2	0.017279	0.009083	1.902	0.951
по убыванию	4	0.015877	0.010900	1.457	0.364
по убыванию	8	0.015846	0.006734	2.353	0.294

Размер массива: 5000000

Данные	Потоки	Tseq, с	Tpar, с	S	E
случайные	1	0.446145	0.422160	1.057	1.057
случайные	2	0.439326	0.216830	2.026	1.013
случайные	4	0.443903	0.223049	1.990	0.498
случайные	8	0.434383	0.209078	2.078	0.260
по возрастанию	1	0.096832	0.091404	1.059	1.059
по возрастанию	2	0.093122	0.052135	1.786	0.893
по возрастанию	4	0.097560	0.055086	1.771	0.443
по возрастанию	8	0.104623	0.055291	1.892	0.237
по убыванию	1	0.108768	0.103608	1.050	1.050
по убыванию	2	0.100524	0.053864	1.866	0.933
по убыванию	4	0.094274	0.052241	1.805	0.451
по убыванию	8	0.092529	0.053236	1.738	0.217

Анализ и вывод

На пяти миллионах случайных чисел при восьми потоках получено ускорение 2.078 и эффективность 0.260. На 10 000 элементов дополнительные потоки не создаются из-за порога 16384; отклонения отношения времён от единицы связаны с малой длительностью, прогревом и шумом измерений.

На больших массивах параллельная обработка половин позволяет сократить время. Линейное ускорение ограничивается последовательным финальным слиянием, созданием и ожиданием потоков, доступом к памяти и разным числом свободных слотов в момент рекурсивного вызова. Упорядоченные и случайные данные отличаются поведением ветвлений при сравнении. Три повтора дают предварительную оценку; результаты зависят от машины и фоновой нагрузки. При одном потоке отдельный замер последовательной сортировки не обязан точно совпадать со временем parallel_sort, которая вызывает тот же последовательный алгоритм.